

E-Commerce DBMS Mini-Project: Part 2 Work Breakdown

Project: E-Commerce Database System

Team Structure: 5 Students

Time Allocation: 4 Hours per Student

Group 1: Core Developers & Code Specialists (3 Students)

This group focuses on implementing, analyzing, and testing the database schema and Python back-end scripts (`schema.sql`, `transactions.py`, `db.py`, and `client.py`).

Student 1: Database Schema & Configuration Lead

- **Allocated Time:** 4 Hours
- **Main Responsibility:** Design and implement the database schema, apply relational constraints, and analyze read-only helper functions.

Detailed Tasks & Deliverables:

1. Schema Implementation (`schema.sql`) – [2 Hours]:

- Write the `CREATE TABLE orders` statement including essential attributes (`order_id`, `user_id`, `total`, `status`, `created_at`) and apply `CHECK` and `DEFAULT` constraints.
- Write the `CREATE TABLE order_items` statement, setting up `FOREIGN KEY` references and ensuring `unit_price` is stored as a snapshot price.
- Implement at least two `CREATE INDEX` statements to improve query performance.

2. Deep-Dive Code Analysis (`db.py`) – [2 Hours]:

- `get_connection()`: Understand database connection setup (PostgreSQL/SQLite) and why `autocommit=False` is used to control transaction boundaries.
- `_query(sql, params)`: Analyze how read-only queries are executed using `RealDictCursor` and how connections are safely closed.
- `fetch_all_products()` & `fetch_product(id)`: Explain how available items and specific product details are fetched from the database.
- `fetch_user(username)` & `fetch_card(user_id)`: Detail how user credentials and gift card balances are verified.

- `fetch_orders_for_user()` & `fetch_order_items()`: Explain how historical orders and itemized line details are retrieved.
-

Student 2: Transaction Logic & Processing Developer

- **Allocated Time:** 4 Hours
- **Main Responsibility:** Implement atomic transactions in `transactions.py` ensuring full ACID compliance during checkout and refund flows.

Detailed Tasks & Deliverables:

1. Transaction Functions Implementation (`transactions.py`) – [3 Hours]:

- `TxResult` **Class:** Understand the response object structure (`success`, `message`, `data`) returned to the caller.
- `checkout(user_id, cart)` **Function:**
 - Validate the shopping cart and serialize data into JSON.
 - Verify product stock availability and ensure the user's card balance covers the total cost.
 - Deduct stock, insert records into `orders` and `order_items`, and handle explicit `COMMIT` and `ROLLBACK` logic.
- `refund_order(user_id, order_id)` **Function:**
 - Verify order ownership and confirm status is `'paid'`.
 - Restore item stock, refund card balances, and mark order status as `'refunded'`.
 - Guarantee single-connection usage with proper error handling and rollbacks.

2. Self-Testing & Verification – [1 Hour]:

- Execute `python transactions.py` and ensure all 4 assertion tests pass successfully.
-

Student 3: CLI Interface & System Integration Tester

- **Allocated Time:** 4 Hours
- **Main Responsibility:** Analyze the CLI user interface, connect transaction logic to user workflows, and execute end-to-end testing.

Detailed Tasks & Deliverables:

1. CLI Code Analysis (`client.py`) – [2 Hours]:

- `login()`: Trace how user logs in (e.g., `alice`, `bob`) query user details via `fetch_user()`.
- `show_products(cart)` & `show_cart(cart, products)`: Explain product menu rendering, price totals, and cart display.
- `checkout_flow()`: Trace user payment confirmation calls to `checkout()` in `transactions.py`.
- `refund_flow()`: Trace order refund choices and execution calls to `refund_order()`.
- `main()` **Loop**: Explain user command parsing for options `[a, r, v, c, h, f, q]`.

2. Database Setup & Integration Testing – [2 Hours]:

- Initialize the database schema using `sqlite3 shop.db < schema.sql` (or PostgreSQL equivalent).
- Perform functional edge-case testing (e.g., purchasing out-of-stock items, insufficient funds, double refunding).

Group 2: Design, AI & Coordination Specialists (2 Students)

This group handles data modeling, answering theoretical database concepts, modern UI expansion, and project submission.

Student 4: Data Modeling Lead (ER Diagram Specialist)

- **Allocated Time:** 4 Hours
- **Main Responsibility:** Analyze project requirements and construct a complete Entity-Relationship (ER) Diagram.

Detailed Tasks & Deliverables:

1. Entity & Attribute Identification – [1 Hour]:

- Identify core entities: `users`, `cards`, `products`, `orders`, and `order_items`.
- Define keys (Primary Keys, Foreign Keys) and essential attributes for all entities.

2. ER Diagram Construction – [2 Hours]:

- Define entity relationships and cardinalities:
 - `users` to `cards` (1-to-1 or 1-to-Many).
 - `users` to `orders` (1-to-Many).
 - `orders` to `products` (Many-to-Many resolved via `order_items` associative entity).

- Draw the visual ER Diagram using tools like Draw.io, Lucidchart, or MySQL Workbench.

3. Validation & Schema Cross-Check – [1 Hour]:

- Verify the ER Diagram against Student 1's `schema.sql` to ensure full structural consistency.

Student 5: Project Coordinator, AI Lead & Modern UI Developer

- **Allocated Time:** 4 Hours
- **Main Responsibility:** Solve in-lab discussion questions, propose a modern user interface prototype, and handle final submission management.

Detailed Tasks & Deliverables:

1. Lab Discussion Questions & AI Research – [1.5 Hours]:

- Research and document comprehensive answers for the lab discussion questions:
 1. *Statement-level commit vs. transaction-level commit*: Explaining loss of Atomicity.
 2. *Concurrent stock purchasing*: SQLite database-level locking vs. PostgreSQL row-level locks.
 3. *Snapshot pricing in `order_items`*: Preventing price history corruption when product catalogue prices update.
 4. *SQLite isolation levels*: Explaining `SERIALIZABLE` isolation and prevented concurrency anomalies.

2. Modern UI Prototype Design – [1 Hour]:

- Design a modern web interface prototype (using **Streamlit** or **Gradio** in Python) to showcase a modern alternative to the CLI interface.
- Present the prototype concept to the team for potential future integration.

3. Project Coordination & Final Submission – [1.5 Hours]:

- Collect all team deliverables (`schema.sql`, `transactions.py`, ER Diagram, and discussion Q&A).
- Review formatting and compliance with coursework requirements and manage final email submission to `sarah.saeed3141@gmail.com`.

Summary Table

Student	Primary Role	Key Deliverable / Focus	Time
Student 1	Schema & DB Lead	schema.sql & db.py analysis	4 Hours
Student 2	Transaction Developer	transactions.py implementation & testing	4 Hours
Student 3	CLI Integration Tester	client.py analysis & End-to-End QA testing	4 Hours
Student 4	Data Modeling Lead	Complete ER Diagram design	4 Hours
Student 5	Coordinator & AI/UI Lead	Q&A Solutions, Web UI Prototype, Email Submission	4 Hours